# Alex Castillo González

## Applied AI Engineer · Python / LLM

Remote — UTC−4 · Spanish (native), English (professional) alex.castillog33@gmail.com · github.com/pcbeingused333 · linkedin.com/in/alexcastillogonzalez · portfolio-alexgonzalez33.vercel.app

## SUMMARY

Applied AI engineer working in Python on retrieval and agent systems, and on the layer that decides whether they survive real users: evaluation and failure handling. My main retrieval project answers over the text of the **GDPR**, built around a constraint regulated domains impose and generic RAG ignores — every statement names the provision it came from, and the system declines when the source does not cover the question. Both projects ship with the harness that measures them, and in each case the harness found defects the tests did not. Fullstack background across Python, TypeScript and Ruby, plus five merged fixes in **Haystack**, deepset's framework for production RAG and agent pipelines, four of them concurrency defects on its async path, and three open fixes to the retrieval evaluation and MMR code in `llama-index-core`.

## SKILLS

**AI / LLM** — Python · LLM APIs (Groq, OpenAI-compatible) · LangChain · LangGraph · ReAct agents and tool use · **Model Context Protocol (MCP)**: building servers and clients · **LLM evaluation**: known-answer datasets, retrieval metrics (hit@1, recall@k, MRR), **abstention and unsupported-claim rate**, agent trajectory scoring, LLM-as-judge for faithfulness / relevancy / correctness · RAG: chunking strategy, retrieval tuning, **structure-aware citation at the provision level** · cross-lingual retrieval

**Regulatory / legal text** — parsing legislative sources into citable provisions (EUR-Lex / Official Journal markup) · citation integrity as a design constraint · evaluating refusal on out-of-corpus questions · GDPR structure (data subject rights, controller and processor obligations, breach notification, administrative fines)

**Data & retrieval** — embeddings (`sentence-transformers`, BGE) · FAISS · PostgreSQL + `pgvector` · PDF ingestion pipelines · structured corpus construction

**Backend & web** — Python · pytest · Ruby on Rails · PostgreSQL · REST APIs · JSON/API integrations (Jira API) · TypeScript · Next.js · React · Tailwind

**Infra & tooling** — **AWS** (Lambda container images, DynamoDB, ECR, IAM least-privilege, CloudWatch) · **Terraform** · **CI/CD with GitHub Actions, OIDC federation** · Docker · Vercel · Git · AI-assisted development (Claude Code, Cursor)

## EXPERIENCE

### Churrería Calderón — Family business · Toronto, Canada

**Oct 2025 – Jul 2026** *(business closed July 2026)*

- **Developer:** built and deployed the business website plus an embeddable AI chat widget answering customer questions grounded only in the business's own information, reusable across clients from a single configuration file. Next.js, React, TypeScript, Tailwind, Groq, Vercel.
- Shipped both before the December 2025 opening, so the site and the assistant were live on day one — while also running day-to-day operations, which is where the judgement about what is worth automating came from.

### Family churrería business — Owner-operator · Catalonia, Spain

**2023 – 2026** *(and in the same business before that)*

- Ran the business single-handed: production, service, customers, purchasing, cash and compliance. No staff to delegate to and no manager to escalate to — if it did not work, it was mine to fix that morning.
- Took it from a mobile trailer to fixed premises, rebuilding the operation around a different site, different hours and a different customer base.

### Le Wagon — Part-time Programming Teacher · Remote (France)

**Oct 2022**

- Taught the new part-time flex cohort of the fullstack bootcamp — Ruby, OOP, SQL and PostgreSQL, HTML/CSS/JavaScript and Ruby on Rails — supporting students through exercises and live debugging.

### TECNOBIT (Grupo Oesía) — Fullstack Developer · Valdepeñas, Spain

**Aug 2022 – Nov 2022** · On-site

- Shipped features into a long-running internal application maintained by a team of five, across a codebase distributed over both GitHub and GitLab.
- **Backend:** Ruby routines pulling data from JSON files and the **Jira API**, transforming and persisting it to PostgreSQL, with multi-stage validation gating document generation and downloads.
- **Frontend:** form-driven pages and PostgreSQL-backed views with role-dependent data, surfacing backend subprocess state so users could follow a download to completion.

## SELECTED PROJECTS

### Business Ops Agent — MCP server + agent, with trajectory evaluation

Live demo · Code

A **Model Context Protocol server** exposing a business's operations (catalog, booking capacity, stock, quoting, orders) as tools any MCP client can call — Claude Desktop, Cursor, or the LangGraph agent bundled with it. The agent carries no business rules; tools are discovered at runtime, so adding one requires no agent change.

- Built an evaluation harness that scores **tool trajectories**, not just answers: which tools were called, in what order, with which arguments, and whether every figure in the reply traces back to a tool result — deterministic, no judge model, so it holds even when a fabricated number happens to be correct. It caught the agent answering with zero tool calls, and caught it **intermittently**, which single-run testing misses. 11/12, mean 0.98.
- **Deployed to AWS** as a remote MCP server — Lambda container behind a Function URL, DynamoDB single-table store, least-privilege IAM, all in Terraform. Storage sits behind an interface, so the same server runs on SQLite locally and DynamoDB in production.
- CI/CD on every push via GitHub Actions authenticating with **OIDC** rather than a stored key, scoped to one branch, and deliberately unable to apply infrastructure.
- Python, MCP 2.0, LangGraph, Groq, AWS (Lambda, DynamoDB, ECR, IAM), Terraform, Docker, GitHub Actions, pytest (161 tests).

### Ask the GDPR — retrieval over regulation, with citations that can be checked

Live demo · Code

LangGraph agent over the full text of the GDPR. Every answer names the provision behind it — `Art. 33(1)`, not a page number — and the system is built to decline when the source does not cover the question. Two modes behind one flag: an in-memory FAISS demo on a free 1 GB container, and a pgvector-backed production path.

- **Made the citation structural rather than incidental.** A regulation is cited by article and paragraph; the page a provision lands on is an artefact of typesetting. So the corpus is not a PDF: a builder parses the Official Journal text from EUR-Lex into **414 provisions** carrying article, paragraph and chapter as metadata, and the chunk size was chosen so **97% of provisions survive as exactly one chunk** — a chunk straddling Art. 33(1) and 33(2) gets attributed to one of them and cites the wrong paragraph.
- **Built an eval for the answers that should never be given.** In legal text a retrieval miss announces itself; an invention is fluent, confident and indistinguishable from a correct answer. So the harness scores refusal against questions the Regulation does not answer but every model has read about — adequacy decisions by country, Schrems II, a CCPA penalty — alongside a deterministic check for citations appearing in the answer but never in the retrieved passages.
- **Re-ran every measurement when the corpus changed, and one result reversed.** Embedding the article heading measurably hurt retrieval under the old embedding model and helped under the new one; carrying the first conclusion forward would have shipped the worse setting on the strength of real evidence. The model swap was worth 8/20 → 13/20 at rank 1 for 42 MB, against a hard 1 GB ceiling.
- CI on every push runs the suite headlessly including a boot test of the app itself — the host redeploys straight from `main`, so the suite is the only gate before the public demo.
- Python, LangChain, LangGraph, Groq, FAISS, pgvector, Streamlit, Docker, GitHub Actions, pytest (72 tests).

**Also** — AI Website Chat Widget (code): drop-in assistant for small-business sites, grounded strictly in the business's own content and reusable for any client from a single config file (Next.js, React, TypeScript, Groq, Vercel). Semantic Recommender: embedding-based recommendations with a feedback loop, as a backend for platforms that have outgrown rule-based filters (Python, `pgvector`, PostgreSQL).

## OPEN SOURCE

**Merged** — five fixes across `deepset-ai/haystack` and its integrations, each with a regression test verified to fail with the fix reverted. Four are concurrency defects: a `User-Agent` rotation cursor shared by every URL of a concurrent fetch, so most retries went out un-rotated (#12364); an async splitter re-splitting over-long chunks through the blocking embedder, on the event loop (#12358); PDF rendering and base64-encoding on the event loop before the first LLM call (#12359); and an `asyncio.Lock` cached across event loops, breaking an OAuth source reused in a new one (#3790). The fifth: three components dropping an init parameter from `to_dict`, so the setting silently reverted to its default whenever the pipeline was saved and reloaded (#3808). Two more merged in `pyfenn/fenn`.

**Open** — three in `llama-index-core` on retrieval evaluation and MMR, and six in Ruby tooling.

**Reported** — found by reading the code, filed with a reproduction: the cached lock above (#3789, triaged `P3`, closed by my PR) and `courrier` #58 — `cc`/`bcc` silently dropped by 8 of the gem's 14 providers.

## EDUCATION

**Le Wagon** — Fullstack Web Development bootcamp, 2022 (Ruby, Rails, JavaScript, SQL/PostgreSQL, HTML/CSS). **Self-directed, 2022–2025** — LaunchSchool coursework and independent study alongside running the business, before moving back into engineering full time.